

LiSA: Lifelong Safety Adaptation via Conservative Policy Induction

Minbeom Kim^{1,2,*}, Lesly Miculicich¹, Bhavana Dalvi Mishra¹, Mihir Parmar¹, Phillip Wallis³, Bharath Chandrasekhar³, Kyomin Jung², Tomas Pfister¹ and Long T. Le¹

¹Google Cloud AI Research, ²Seoul National University, ³Google

As AI agents move from chat interfaces to autonomous systems that read private data, call tools, and execute multi-step workflows, guardrails become an important line of defense against concrete deployment harms. In these settings, guardrail failures are no longer merely answer-quality errors: they can leak secrets, authorize unsafe actions, or block legitimate work. The hardest failures are often contextual: whether an action is acceptable depends on local privacy norms, organizational policies, and user expectations that resist pre-deployment specification. This creates a practical gap: guardrails must adapt to their own operating environments, yet deployment feedback is typically limited to sparse, noisy user-reported failures, and repeated fine-tuning is often impractical. To address this gap, we propose LiSA (Lifelong Safety Adaptation), a conservative policy induction framework that improves a fixed base guardrail through structured memory. LiSA converts occasional failures into reusable policy abstractions so that sparse reports can generalize beyond individual cases, adds conflict-aware local rules to prevent overgeneralization in mixed-label contexts, and applies evidence-aware confidence gating via a posterior lower bound, so that memory reuse scales with accumulated evidence rather than empirical accuracy alone. Across PrivacyLens+, ConFaide+, and AgentHarm, LiSA consistently outperforms strong memory-based baselines under sparse feedback, remains robust under noisy user feedback even at 20% label-flip rates, and pushes the latency–performance frontier beyond backbone model scaling. Ultimately, LiSA offers a practical path to secure AI agents against the unpredictable long tail of real-world edge risks.

1. Introduction

Large language models (LLMs) increasingly power AI agents that do more than answer questions: they access private data (Abaev et al., 2026), call privileged tools (Drouin et al., 2024), and execute multi-step workflows (Yao et al., 2025). As such systems move into deployment, the cost of error escalates from low-stakes generation mistakes to concrete harms: incorrect allow decisions can leak private information or authorize unsafe actions, while incorrect refusals can block legitimate work. To mitigate these risks, deployed AI systems increasingly rely on safety guardrails as a critical last line of defense.

A growing line of work (Inan et al., 2023; Kim et al., 2026; Li et al., 2025; Luo et al., 2025; Zeng et al., 2024) has introduced different guardrails, including refusal-oriented prompting, safety classifiers, rule-based validators, and runtime monitors. However, these methods share a fundamental limitation: they rely on a static, general-purpose definition of harm specified before deployment. In practice, safety and privacy boundaries are rarely universal. Acceptable behavior is shaped by local organizational rules, shifting user expectations, and task-specific risk tolerances that are difficult to fully enumerate in advance (Gomaa et al., 2026; Ravichandran et al., 2026; Yi et al., 2025). Consequently, a fixed guardrail is often mismatched to its unique deployment environment—leaving it too permissive against novel risks while remaining too restrictive for legitimate, context-specific actions.

Corresponding author(s): Minbeom Kim: minbeomkim@snu.ac.kr and Long T. Le: longtle@google.com

* This work was done while Minbeom was a student researcher at Google Cloud AI Research.

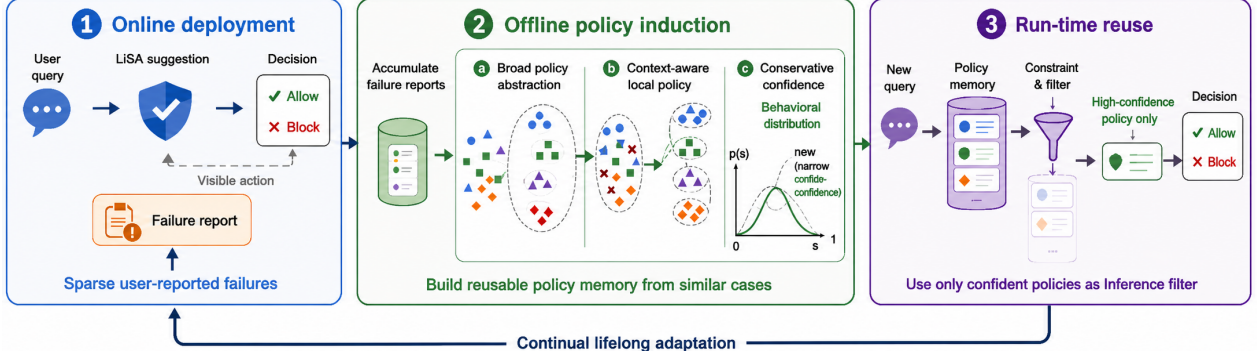


Figure 1 | **Overview of LiSA guardrail.** 1) *Online (left)*: a fixed base guardrail decides on each query and logs user-reported mistakes. 2) *Offline (center)*: sparse reports are abstracted into broad policy items, while mixed-label neighborhoods are rendered into conflict-aware local refinement rules. Broad policy items are scored by a Beta posterior over their support/contradiction counts. 3) *Run-time reuse (right)*: semantically matched local rules are surfaced as narrow refinement cues, and broad policies are surfaced only when their posterior lower bound clears a label-specific threshold.

To bridge this gap, we formulate the problem of **deployment-time guardrail adaptation**: a deployed guardrail should improve over time from the failures that arise in its own operating context. This setting imposes three constraints that distinguish it from standard supervised updating. First, adaptation must occur under *sparse supervision* (Zhang et al., 2024): users rarely provide dense, curated labels, yielding only occasional corrections. Second, feedback can be *noisy* (Kim et al., 2021): users may disagree, misattribute failures, or report preferences as safety concerns. Third, adaptation must remain *conservative* (Soltani et al., 2013): overgeneralizing from a handful of local mistakes can degrade helpfulness through overly broad refusals, or compromise safety by over-trusting weakly supported permissive rules.

To address these challenges, we propose **LiSA** (Lifelong Safety Adaptation), a conservative policy induction framework organized as an online–offline loop. Rather than repeatedly fine-tuning the base guardrail, LiSA improves it through structured policy memory and evidence-aware reuse as in Figure 1. Broad policy abstractions turn sparse failure reports into reusable guidance; conflict-aware local policies preserve fine-grained resolution near mixed-label regions; and confidence-gated reuse surfaces broad memory only when accumulated evidence supports it, preventing weakly supported abstractions from influencing inference too early. Together, these components allow a fixed guardrail to adapt to its operating environment while remaining stable under sparse, noisy feedback.

Empirically, we evaluate LiSA on PrivacyLens+ (Shao et al., 2024), ConFaide+ (Miresghallah et al., 2024), and AgentHarm (Andriushchenko et al., 2025) under simulated deployment streams with sparse failure reports. Across datasets and two lightweight online guardrails, LiSA consistently outperforms the fixed base guardrail and strong memory-based baselines. Ablations reveal that while broad abstraction aids adaptation—much like existing memory baselines—local policies drive the most substantial performance improvements. Furthermore, confidence gating stabilizes these gains and renders the system highly robust even when reported labels are noisy. Finally, our latency analysis shows that structured memory offers a more efficient path than simply scaling the guardrail: LiSA attached to the lightweight model pushes the latency–performance frontier beyond larger un-adapted backbones.

Contributions. Our contributions are threefold:

- We formulate the problem of *lifelong guardrail adaptation*, where a fixed base guardrail improves from sparse, potentially noisy user-reported corrections without repeated fine-tuning.
- We propose **LiSA**, a structured policy memory framework driven by three core mechanisms: broad policy abstraction for sparse reuse, conflict-aware local refinement for mixed-label regions, and conservative confidence-gated reuse that surfaces memory only when accumulated evidence warrants it.
- Across PrivacyLens+, ConFaide+, and AgentHarm, we demonstrate that LiSA: (i) consistently outperforms strong memory-based baselines under sparse feedback; (ii) remains robust to noisy reports, with conservative confidence-gated reuse identified as a key stabilizing factor; (iii) improves boundary-sensitive decisions through conflict-aware local refinement; and (iv) pushes the latency–performance frontier beyond base-model scaling, showing that memory-based adaptation can be a more efficient route to improving deployed guardrails than using larger static backbones.

2. Related works

2.1. Guardrails for AI agent safety

A broad family of guardrailing mechanisms has emerged as LLMs gain access to private data and privileged tools, including general-purpose safety classifiers (Han et al., 2024; Inan et al., 2023; Liu et al., 2025; Zeng et al., 2024), implicit toxicity detectors (Kim et al., 2024), refusal-oriented safeguards (Arditi et al., 2024; Yuan et al., 2025), monitors over agent reasoning or trajectories (Korbak et al., 2025), and defenses against indirect prompt manipulation (Kim et al., 2026; Leng et al., 2025; Li et al., 2025; Xiang et al., 2025). While these methods address complementary risk surfaces, they are typically specified before deployment and remain largely fixed during use. When out-of-distribution failures emerge in real environments, a natural response is to collect more examples and update the guardrail through retraining (Tamang et al., 2025). This path is mismatched to the regime we study, where failures are sparse, feedback arrives as occasional user reports, and repeated training is often impractical (Wickramasinghe et al., 2023). We therefore ask whether a fixed base guardrail can improve directly from deployment-time experience, without retraining, by organizing sparse feedback into lightweight reusable structure.

2.2. Memory and policy abstraction for adaptive agents

A growing body of work equips LLM agents with memory (Hu et al., 2025) so that they can accumulate experience, either by retrieving past trajectories as exemplars (Zheng et al., 2024) or by inducing reusable natural-language policies, codes (Lee et al., 2025b), or reflections from prior outcomes (Min et al., 2026; Ouyang et al., 2026; Shinn et al., 2023; Tan et al., 2025). These methods are typically developed for reasoning and planning settings (Choi et al., 2026; Hu et al., 2025), where supervision comes from task success and a poorly surfaced memory item degrades answer quality rather than causing a direct safety failure; broad abstraction and relatively permissive retrieval are reasonable defaults in that regime.

Guardrailing departs from this regime in several ways that matter for memory design. Labels are contextual (Yi et al., 2025) and often user- or organization-specific (Abaev et al., 2026; Sanyal et al., 2025) rather than determined by task success, so induced policies inherit feedback noise directly. Moreover, a single mis-surfaced memory item can trigger a privacy leak or an unsafe allow decision, making weakly supported reuse much riskier than in task-oriented memory systems. These properties

make broad abstraction useful but also make naive retrieval substantially more brittle in the guardrail setting.

Within safety guardrails domain, adaptive memory remains relatively underexplored. AGrail (Luo et al., 2025) maintains an updatable safety checklist, but checklist-style adaptation has limited resolution in mixed-label regions and does not explicitly calibrate memory reuse by confidence. Recent works study personalized guardrails (Wu et al., 2025a,b) by conditioning safety reasoning on user profiles, but focus on profile-conditioned decision making rather than learning from sparse case-level corrections. LiSA targets this complementary deployment-time adaptation setting by jointly addressing two failure modes of prior adaptive memory: it adds conflict-aware local refinement so that mixed-label neighborhoods are not collapsed into a single broad rule, and it gates broad-policy reuse with a Beta-posterior lower bound rather than relying on retrieval similarity or empirical accuracy alone.

3. LiSA guardrails

3.1. Problem setup and deployment loop

We study deployment as an alternating *online–offline* loop. Online, the guardrail receives a stream of deployment inputs $x_t \in \mathcal{X}$, and outputs a binary decision $\hat{y}_t \in \{0, 1\}$, where 0 denotes ALLOW and 1 denotes REFUSE. A fixed base guardrail

$$G_{\text{base}} : \mathcal{X} \rightarrow \{0, 1\}$$

is available throughout deployment. As the system is used, it accumulates sparse user-reported corrections $\mathcal{B}_t = \{(x_i, \tilde{y}_i)\}_{i=1}^{n_t}$. These reports arrive irregularly and may be noisy. Rather than repeatedly fine-tuning the guardrail, we periodically refresh memory from the accumulated reports and redeploy the updated memory in the next online phase. This yields a lightweight form of *lifelong safety adaptation*: the deployed guardrail improves over time while the base guardrail remains fixed.

Our method combines three components: broad policy memory for reusable coverage (§3.2), conflict-aware local policies for ambiguous regions (§3.3), and confidence-gated reuse (§3.4) so that broad abstractions are surfaced only when sufficiently supported, while local rules are used as narrow refinement cues for semantically close mixed-label cases.

3.2. Structured policy memory

The central unit of adaptation is a *policy item*. At each offline refresh, LiSA converts newly reported failures into broad policy candidates and merges semantically overlapping candidates across refreshes. A broad policy item is represented as

$$m = (r_m, \ell_m, \nu_m),$$

where r_m is a natural-language policy statement, $\ell_m \in \{0, 1\}$ is the label it recommends, and ν_m stores metadata such as provenance, examples, and runtime statistics. For instance, a broad item induced during deployment may read “*Sharing general or public information is appropriate even by professionals in confidential roles,*” with $\ell_m = \text{ALLOW}$ and ν_m aggregating support and contradiction counts across the reports that induced it (Appendix D.3, Example 1). These items are designed for sparse reuse: rather than storing each failure only as an isolated case, LiSA stores a compact abstraction that can guide future decisions in related contexts. The resulting broad memory is

$$\mathcal{M}_t = \{m_j\}_{j=1}^{M_t}.$$

Why broad abstraction alone misses local boundaries. Broad memory improves reuse under sparse feedback, but it can also become too coarse. Nearby contexts with different labels may be covered by the same broad policy, causing the memory to overgeneralize across a local decision boundary. Since sparse feedback does not support refining every broad policy, LiSA adds conflict-aware local policies only in mixed-label regions where broad reuse is most likely to fail. Section 3.5 formalizes this motivation, and Appendix C.4 gives the operational refresh procedure.

3.3. Conflict-aware local policies

When the report neighborhood associated with a broad pattern contains both labels with non-trivial support, we treat that region as evidence that broad reuse is overgeneralizing across a local boundary. For instance, coworker-to-coworker sharing may be appropriate for routine coordination but inappropriate when it involves client insurance information without clear need-to-know authorization (Nissenbaum, 2004). We then induce one or more narrower policy items

$$e = (r_e, \ell_e, \nu_e),$$

and store them in local memory

$$\mathcal{L}_t = \{e_k\}_{k=1}^{L_t}.$$

As an instance, a region centered on “a friend attended a public lecture” splits between ALLOW and REFUSE depending on whether the lecture is a public talk or a fringe event; LiSA renders complementary label-specific cues for this region rather than forcing a single broad rule across the boundary (Appendix D.3, Example 3). Broad and local policies are stored as natural-language memory entries rather than executable rules, but they play distinct roles and are governed by different reuse rules. Broad memory provides reusable coverage under sparse feedback and is therefore subject to confidence gating (Section 3.4), so that weakly supported abstractions do not influence inference too early. Local memory, by contrast, is induced *only* in mixed-label regions where nearby cases split labels; its purpose is to expose a contradictory boundary cue to the inference model rather than to assert a globally reusable rule. Because a local rule is, by construction, anchored to a conflict-heavy semantic neighborhood, applying the same broad-policy gate to it would suppress exactly the boundary signal it is meant to surface. We therefore do *not* gate local rules at inference time: any retrieved local rule is surfaced together with its support and contradiction counts as evidence for the inference model. Appendix C.4 specifies the deterministic procedure that detects mixed-label regions and renders label-specific local rules.

3.4. Confidence-gated online guardrail

Let

$$Q_t = \mathcal{M}_t \cup \mathcal{L}_t$$

denote the full memory at deployment time. For each broad item $m \in \mathcal{M}_t$, we maintain support and contradiction counts (s_m, c_m) , initialized from the inducing reports and updated when surfaced broad items later receive additional feedback. Local items also store support and contradiction counts, but these counts are serialized as local evidence rather than used for confidence gating.

We model the transfer reliability of a broad policy item by a latent accuracy $\theta_m \in [0, 1]$, the probability that the item remains correct when surfaces on a future case. With a uniform prior,

$$\theta_m \sim \text{Beta}(1, 1),$$

the posterior after observing support and contradiction counts is

$$\theta_m \mid \text{data} \sim \text{Beta}(1 + s_m, 1 + c_m).$$

We define confidence as the lower δ -quantile of this posterior,

$$\text{Conf}(m) = Q_\delta(\text{Beta}(1 + s_m, 1 + c_m)). \quad (1)$$

so the confidence score in Eq. 1 reflects both empirical correctness and evidence volume. Weakly tested broad items therefore remain cautious, while repeatedly validated broad items are trusted more strongly. Proposition B.2 gives the resulting posterior error-budget guarantee, and Appendix B.6 motivates the Beta choice over variance-oblivious alternatives.

At inference time, the system retrieves small candidate sets from broad and local memory by semantic similarity, filters only the retrieved broad items using label-sensitive confidence thresholds, serializes the surviving broad items together with retrieved local rules into a structured guardrail prompt, and asks the inference model to output the final decision. If no broad item survives filtering and no local rule is retrieved, the system falls back to the base guardrail $G_{\text{base}}(x_t)$.

We use separate thresholds τ_{refuse} and τ_{allow} for refusal-oriented and allow-oriented broad memory. A retrieved broad item m is surfaced only if

$$\text{Conf}(m) \geq \tau(\ell_m), \quad \tau(\ell_m) = \begin{cases} \tau_{\text{refuse}}, & \ell_m = 1, \\ \tau_{\text{allow}}, & \ell_m = 0. \end{cases} \quad (2)$$

This gating rule in Eq. 2 provides a practical operating knob for broad-policy reuse: higher thresholds make the system more conservative, while asymmetric thresholds allow safety- or utility-prioritized deployment without changing the base guardrail.

Why conservative confidence rather than empirical accuracy alone. In sparse-feedback regimes, empirical accuracy can overstate the reliability of weakly tested broad items: a policy validated once and a policy validated many times may both appear perfect. Using a posterior lower bound avoids surfacing such brittle broad memory too early, while still allowing repeatedly validated broad items to influence inference more strongly.

3.5. Formal design rationale

The preceding components are motivated by two standard observations about sparse adaptive decision making. We include them here only to clarify where LiSA allocates refinement and when it reuses memory; Appendix B gives the corresponding formal statements.

Refine broad states with label conflict. Broad policy memory is useful because it lets sparse reports generalize beyond individual cases, but its main failure mode is collapsing nearby cases with different labels into the same reuse state. For a broad state z , let $\eta_B(z) = \Pr(Y = 1 \mid Z = z)$ denote the REFUSE rate within z . If one broad decision covers all cases in z , refinement can only recover errors on the minority label, whose total mass is

$$\Pr(Z = z) \min\{\eta_B(z), 1 - \eta_B(z)\}. \quad (3)$$

This product is zero for label-pure states and largest when both labels have substantial support. This motivates using local policies selectively in mixed-label regions, rather than refining every broad abstraction.

Gate reuse by evidence, not empirical accuracy alone. Sparse feedback also makes newly induced broad memory look more reliable than it is. A broad policy item with one support and no contradiction has empirical accuracy 1.0, but little evidence. LiSA therefore scores each broad item m with support and contradiction counts (s_m, c_m) using the lower posterior quantile

$$\text{Conf}(m) = Q_\delta(\text{Beta}(1 + s_m, 1 + c_m)).$$

The reuse rule $\text{Conf}(m) \geq \tau(\ell_m)$ keeps weakly tested broad items from influencing inference too early, while allowing repeatedly validated broad items to be surfaced. This is not an end-to-end correctness guarantee for the prompted guardrail, but an evidence-sensitive criterion for controlling broad-memory reuse under sparse and noisy feedback.

3.6. Lifelong online–offline adaptation

The system alternates between online deployment and offline memory refresh. Online, current memory guards new inputs; offline, accumulated reports are folded back into memory by rebuilding broad items, regenerating local items in mixed-label regions, and updating broad-memory confidence statistics. The refreshed memory is redeployed in the next round, enabling continual adaptation without repeated fine-tuning. Algorithm 1 summarizes the LiSA online–offline adaptation procedure.

Why global refresh rather than append-only growth. New reports do not merely add rules; they can reveal that existing items are redundant, overly broad, or near a previously unseen mixed-label boundary. Append-only updates would therefore accumulate overlapping abstractions and make memory increasingly order-dependent. We instead rebuild the policy set from the cumulative report bank so that broad items can be re-merged, conflict-heavy regions re-identified, and local refinements regenerated under the full evidence. In practice, only newly reported failures incur LLM-based policy induction; existing items are re-clustered and merged deterministically over their stored statements and statistics, so the refresh cost scales with new reports rather than the size of accumulated memory.

Preserving runtime evidence across refresh. A key challenge in rebuilding memory is handling online support and contradiction counts. Discarding them wastes deployment evidence, but transferring them across semantically similar yet distinct abstractions is unreliable. We therefore carry over runtime statistics only for broad policy statements that survive the rebuild, keeping confidence estimates meaningful without propagating evidence beyond the items that originally collected it.

4. Experimental setting

We evaluate lifelong guardrail adaptation from sparse user-reported failures: the base guardrail remains fixed, feedback is provided only for misclassified inputs, and memory is refreshed periodically rather than through repeated fine-tuning. This setup allows us to investigate whether structured policy memory can improve a lightweight guardrail over time, and whether the conservative mechanisms from Section 3.5—local refinement in mixed-label regions and evidence-gated broad reuse—behave as intended in practice. We organize the experiments around four questions:

RQ1 (Sparse adaptation). Can a fixed base guardrail improve over deployment rounds using only sparse failure reports?

RQ2 (Component roles). Do local refinement and confidence-gated reuse contribute in the distinct ways suggested by the design rationale?

Algorithm 1 LiSA: Lifelong Safety Adaptation**Require:** Base guardrail G_{base} , Memory $\mathcal{B}$, Broad policies $\mathcal{M}$, Local policies $\mathcal{L}$

```

1: Initialize  $\mathcal{B}, \mathcal{M}, \mathcal{L} \leftarrow \emptyset$ 
2: for each deployment round do
    // Online phase: guard each input with current memory
3:   for each input  $x_t$  in the round do
4:      $\mathcal{M}_{\text{Ret}} \leftarrow \text{Retrieve}(x_t, \mathcal{M}), \mathcal{L}_{\text{Ret}} \leftarrow \text{Retrieve}(x_t, \mathcal{L})$            ▶ retrieve broad / local policies
5:      $\mathcal{M}_{\text{Ret}} \leftarrow \{ m \in \mathcal{M}_{\text{Ret}} : \text{Conf}(m) \geq \tau(\ell_m) \}$            ▶ confidence gating
6:     if  $\mathcal{M}_{\text{Ret}} \cup \mathcal{L}_{\text{Ret}} = \emptyset$  then
7:        $\hat{y}_t \leftarrow G_{\text{base}}(x_t)$            ▶ if no policy applies → fall back to base decision
8:     else
9:        $\hat{y}_t \leftarrow G_{\text{base}}(x_t, \mathcal{M}_{\text{Ret}} \cup \mathcal{L}_{\text{Ret}})$            ▶ decision with retrieved policies
10:    end if
11:    Append correction  $(x_t, \tilde{y}_t)$  to  $\mathcal{B}$  if any           ▶ collect new sparse feedback
12:  end for
    // Offline phase: refresh memory from accumulated reports
13:   $\mathcal{M} \leftarrow \mathcal{M} \cup \text{InduceBroad}(\mathcal{B})$            ▶ abstract new failures into broad policy candidates
14:   $\mathcal{M} \leftarrow \text{Cluster}(\mathcal{M})$            ▶ merge similar items; carry over evidence ( $s_m, c_m$ )
15:   $\mathcal{L} \leftarrow \text{InduceLocal}(\mathcal{B})$            ▶ regenerate boundary rules in mixed-label regions
16: end for

```

RQ3 (Robustness). Is adaptation stable when reported labels are noisy, and is this stability specifically tied to evidence-aware confidence measurement?

RQ4 (Cost vs. scaling). Does structured memory provide a better cost–performance trade-off than simply using a larger un-adapted guardrail?

Sections 5.1–5.4 address these questions in order.

4.1. Datasets

We evaluate on three binary guardrailing benchmarks with complementary risk profiles: **PrivacyLens+** (Shao et al., 2024), **ConFaide+** (Miresghallah et al., 2024), and **AgentHarm** (Andriushchenko et al., 2025). The “+” variants of PrivacyLens and ConFaide include ambiguous contextual cases introduced by Yi et al. (2025). These privacy benchmarks are useful for evaluating local decision boundaries because labels can change under subtle contextual differences. AgentHarm broadens the evaluation beyond privacy to harmful agent behavior. We map all datasets into a shared binary decision space, ALLOW and REFUSE. When multiple examples are derived from the same base scenario, we keep them in the same split to avoid overlap between adaptation and held-out evaluation.

Deployment simulation. We simulate deployment over N days. On each day, the guardrail predicts decisions for a stream of cases. At the end of the day, each adaptive method receives feedback only on the cases it misclassified, paired with the reported label, and updates its memory before the next day. Held-out evaluation is performed after each update on a fixed test set that is never used for adaptation. For noisy-feedback experiments, each reported failure label is independently flipped with probability ρ (i.e., noise ratio) before being passed to the adaptive method.

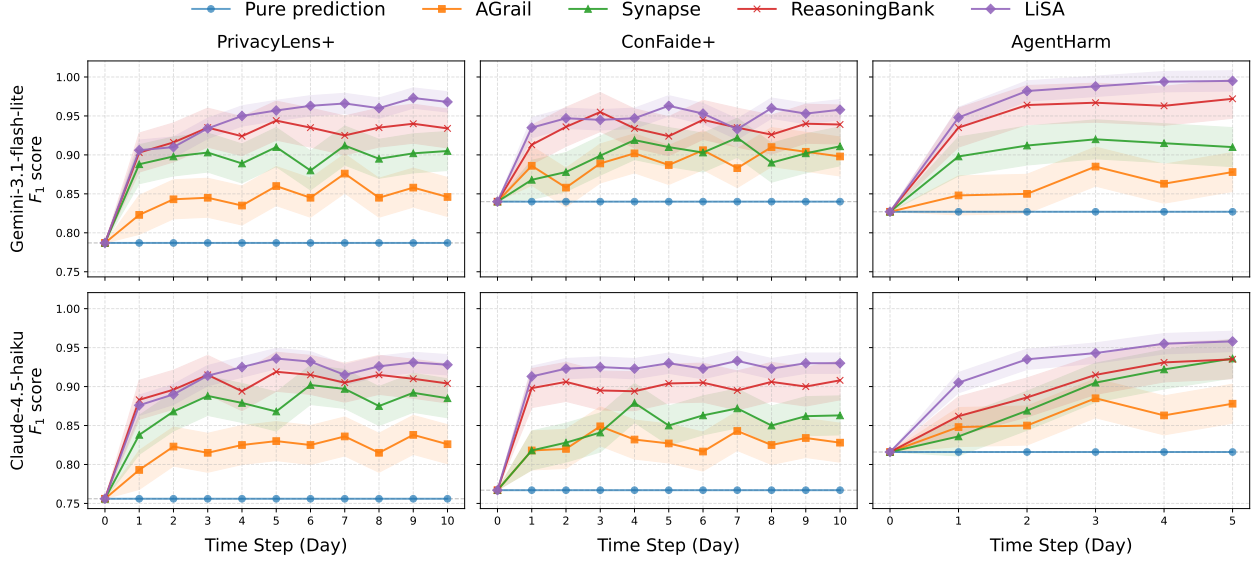


Figure 2 | **Lifelong safety adaptation results.** Held-out F1 versus deployment day with sparse failure reports. Each curve is averaged over five seeds, and shaded regions indicate standard deviation. Upper row: Gemini-3.1-flash-lite as the online guardrail; lower row: Claude-Haiku-4.5.

4.2. Baselines

We compare LiSA against four baselines. **Pure Prediction** uses the base guardrail without adaptation. **AGrail** (Luo et al., 2025) updates a test-time checklist from observed failures and a previous history. **Synapse** (Zheng et al., 2024) stores corrected cases and retrieves similar past examples at inference time. **ReasoningBank** (Ouyang et al., 2026) converts failures into reusable natural-language memories and retrieves them as policy-like guidance. In this setting, ReasoningBank serves as a broad-policy-only memory baseline: it tests whether policy abstraction alone is sufficient, without LiSA’s conflict-aware local refinement or confidence-gated surfacing. All adaptive methods receive only their own day-end failure reports and never observe dense labels for the full stream. Synapse and ReasoningBank were originally designed for reasoning tasks; Appendix C.5 describes how we adapt them to this guardrail setting.

Implementation details We instantiate the online guardrail with two lightweight models, Gemini-3.1-flash-lite and Claude-Haiku-4.5, to test whether the adaptation pattern depends on a particular model family. For both ReasoningBank and LiSA, the offline manager for policy induction is Gemini-3.1-pro, and semantic retrieval uses Gemini-embedding-001 (Lee et al., 2025a). These offline components are kept fixed across online guardrail configurations. Adaptive methods use fixed prompt templates across datasets. We report accuracy and macro-F1 on the same held-out split across methods, averaged over five seeds. Additional split sizes, retrieval limits, thresholds, prompts, and noise construction details are provided in Appendix C.

5. Empirical results

5.1. Main results: lifelong adaptation from sparse failure reports

Figure 2 shows held-out macro-F1 over deployment days. The fixed **Pure Prediction** baseline serves as the un-adapted reference performance. Memory-based methods improve over this baseline, but the type of memory matters. **AGrail** and **Synapse** yield limited gains, suggesting that checklist-style

summaries or isolated case reuse do not provide enough transferable structure under sparse feedback. **ReasoningBank** performs better, indicating that broad policy abstraction is a useful unit of lifelong adaptation.

LiSA achieves the strongest performance across all three benchmarks and under both online guardrail models. The gain over ReasoningBank is moderate but consistent, which is the expected pattern if broad abstraction already captures much of the reusable signal, while LiSA’s additional mechanisms mainly address the failure modes of broad reuse. This matches the design rationale in Section 3.5: conflict-aware local rules help in regions where semantically similar cases have different labels, and confidence-gated surfacing limits the influence of broad policies that have not yet accumulated enough support. The next subsection isolates each component empirically.

5.2. Component roles: local refinement improves boundaries, gating stabilizes reuse

Table 1 separates the two additions LiSA makes on top of broad policy memory, in the noise-free regime ($\rho = 0\%$). The largest mean drop comes from removing local rules. This is consistent with the conflict-mass theoretical prediction in Eq. 3: when nearby cases split labels, a single broad policy must cover both sides of a local boundary, and narrower rules provide the additional resolution needed in these regions. Removing confidence gating, by contrast, has only a small effect on the mean but noticeably increases seed variance; the same pattern becomes stronger when both mechanisms are removed.

Table 1 | **Ablation study of LiSA.** Final-day macro-F1 and standard deviation across five seeds, averaged across benchmarks with Gemini-3.1-flash-lite.

Ablation	F1
LiSA	0.962 (± 0.013)
w/o Conf gate	0.959 (± 0.025)
w/o Local rules	0.931 (± 0.021)
w/o both	0.925 (± 0.038)

In conclusion, local refinement drives the boundary-level F1 gain, and confidence gating stabilizes this improvement. The latter effect becomes more pronounced once reported labels are noisy, as we show in Section 5.3.

5.3. Robustness under noisy user reports and its source

Table 2 evaluates adaptation when reported failure labels are corrupted before memory refresh. The results reveal a trade-off between transfer and noise sensitivity. Broad abstraction transfers well under clean feedback: **ReasoningBank** is the strongest baseline at $\rho = 0\%$. Under noise, however, a mislabeled report can become reusable guidance and affect many later decisions, causing performance to drop sharply. Direct case retrieval, as in **Synapse**, localizes such errors and degrades more gradually, but transfers less effectively and has lower clean-feedback performance.

LiSA retains most of its clean-feedback gain under both noise levels. This indicates that it preserves the transfer benefits of broad abstraction without letting weak policies dominate inference too early. LiSA does not identify noisy reports directly; rather, confidence-gated reuse requires broad policies to accumulate evidence before surfacing, while contradictions lower their posterior confidence. This provides an evidence-sensitive reuse mechanism that is more stable than unconditional retrieval, consistent with the posterior surfacing rule in Section 3.4.

Confidence measurement as the stabilizing factor. Table 3 isolates the confidence mechanism at $\rho = 20\%$. All variants use the same broad and local memory; they differ only in how retrieved broad items are surfaced. Without gating, performance drops substantially, showing that local refinement

Table 2 | **Noise robustness.** Final-day F1 score averaged across benchmarks (five seeds) with Gemini-3.1-flash-lite. ρ : label-flip ratio.

Method	$\rho=0\%$	$\rho=10\%$	$\rho=20\%$
Pure Prediction	0.818	0.818	0.818
AGrail	0.857	0.846	0.832
Synapse	0.891	0.878	0.863
ReasoningBank	0.936	0.882	0.857
LiSA	0.962	0.933	0.917

Table 3 | **Confidence measurement ablation.** Final-day macro-F1 at $\rho=20\%$, same setting as left. All variants share LiSA’s two-level memory and differ only in how retrieved broad items are gated.

Confidence	$\rho=20\%$
No gating	0.854 (± 0.044)
Accuracy $s/(s+c)$	0.883 (± 0.032)
Beta Quantile	0.917 (± 0.025)

alone cannot prevent noisy broad policies from being reused without sufficient evidence. Empirical accuracy gating helps, but it ignores evidence volume: a policy with one support and no contradiction and a policy with many supports and no contradictions both receive accuracy 1.0. The Beta lower quantile separates these cases, blocking weakly tested broad policies early while allowing repeatedly validated policies to be reused. This makes it the most stable reuse rule among the tested variants.

5.4. Latency–performance trade-off

Finally, we compare scaling a static guardrail with adapting a smaller one. Figure 3 plots final-day macro-F1 against average inference latency relative to Gemini-3.1-flash-lite. Larger un-adapted backbones trace the usual scaling frontier: higher F1 at higher per-query latency. Adaptive methods behave differently. **AGrail** remains below this frontier, as checklist reasoning adds latency without commensurate gains under sparse feedback. In contrast, memory-based reuse shifts the frontier upward: **Synapse** benefits from direct case reuse, **ReasoningBank** from policy abstraction, and **LiSA** moves closest to the oracle by adding conflict-aware local refinement and confidence-gated reuse.

Memory-based adaptation has a different cost structure from backbone scaling. Offline refresh is triggered only by sparse reported failures, runs outside the per-query serving path, and amortizes over many later decisions once the induced policies are reused. Detailed offline token costs are reported in Appendix D.1. Thus, in sparse-feedback guardrail deployment, structured memory offers a more efficient path to stronger decisions than scaling a backbone guardrail alone.

6. Conclusion

We studied lifelong safety adaptation under sparse, noisy user-reported failures. Our results support a simple claim: effective adaptation does not require fine-tuning or larger models, but does require calibrated reuse of deployment experiences. LiSA realizes this thesis through structured policy memory: abstractions generalize sparse failures, local rules preserve boundary cues in mixed-label regions, and confidence gating prevents over-generalization of weakly supported memory. Across benchmarks, this conservative memory-driven design improves guardrail performance, remains robust to noisy feedback, and pushes the latency–performance frontier beyond base-model scaling.

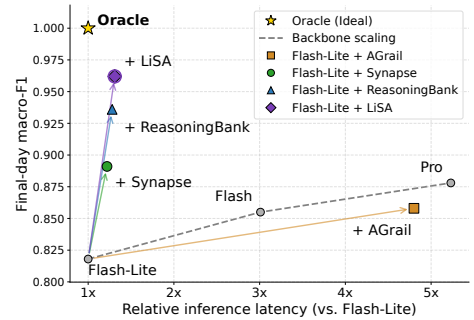


Figure 3 | **Latency–F1 trade-off.** Memory-based adaptation pushes the frontier beyond base-model scaling, with LiSA closest to the oracle.

More broadly, LiSA suggests a direction for future guardrails: they should adapt to deployment experience, but only through evidence-calibrated reuse. Static guardrails cannot anticipate the long tail of local norms and evolving user expectations; yet unconstrained adaptation can introduce over-refusal or unsafe permission. Conservative policy induction offers a practical middle ground, allowing guardrails to learn from their operating environments while preserving evidence-grounded caution.

References

- N. Abaev, D. Klimov, G. Levinov, D. Mimran, Y. Elovici, and A. Shabtai. Agentguardian: Learning access control policies to govern ai agent behavior. *arXiv preprint arXiv:2601.10440*, 2026.
- M. Andriushchenko, A. Souly, M. Dziemian, D. Duenas, M. Lin, J. Wang, D. Hendrycks, A. Zou, J. Z. Kolter, M. Fredrikson, Y. Gal, and X. Davies. Agentharm: A benchmark for measuring harmfulness of LLM agents. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=AC5n7xHuR1>.
- Anthropic. Introducing claude haiku 4.5, 2025. URL <https://www.anthropic.com/news/claude-haiku-4-5>.
- A. Arditi, O. Obeso, A. Syed, D. Paleka, N. Panickssery, W. Gurnee, and N. Nanda. Refusal in language models is mediated by a single direction. *Advances in Neural Information Processing Systems*, 37: 136037–136083, 2024.
- J. Choi, J. Yoon, L. T. Le, S. Jha, and T. Pfister. Policybank: Evolving policy understanding for llm agents. *arXiv preprint arXiv:2604.15505*, 2026.
- A. Drouin, M. Gasse, M. Caccia, I. H. Laradji, M. Del Verme, T. Marty, D. Vazquez, N. Chapados, and A. Lacoste. WorkArena: How capable are web agents at solving common knowledge work tasks? In R. Salakhutdinov, Z. Kolter, K. Heller, A. Weller, N. Oliver, J. Scarlett, and F. Berkenkamp, editors, *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 11642–11662. PMLR, 21–27 Jul 2024. URL <https://proceedings.mlr.press/v235/drouin24a.html>.
- A. Gomaa, A. Salem, and S. Abdelnabi. Converse: Benchmarking contextual safety in agent-to-agent conversations. In *Findings of the Association for Computational Linguistics: EACL 2026*, pages 3246–3268, 2026.
- S. Han, K. Rao, A. Ettinger, L. Jiang, B. Y. Lin, N. Lambert, Y. Choi, and N. Dziri. Wildguard: Open one-stop moderation tools for safety risks, jailbreaks, and refusals of llms. *Advances in neural information processing systems*, 37:8093–8131, 2024.
- Y. Hu, S. Liu, Y. Yue, G. Zhang, B. Liu, F. Zhu, J. Lin, H. Guo, S. Dou, Z. Xi, et al. Memory in the age of ai agents. *arXiv preprint arXiv:2512.13564*, 2025.
- H. Inan, K. Upasani, J. Chi, R. Rungta, K. Iyer, Y. Mao, M. Tontchev, Q. Hu, B. Fuller, D. Testuggine, et al. Llama guard: Llm-based input-output safeguard for human-ai conversations. *arXiv preprint arXiv:2312.06674*, 2023.
- C. D. Kim, J. Jeong, S. Moon, and G. Kim. Continual learning on noisy data streams via self-purified replay. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 537–547, 2021.

- M. Kim, J. Koo, H. Lee, J. Park, H. Lee, and K. Jung. Lifetox: Unveiling implicit toxicity in life advice. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 2: Short Papers)*, pages 688–698, 2024.
- M. Kim, M. Parmar, P. Wallis, L. Miculicich, K. Jung, K. D. Dvijotham, L. T. Le, and T. Pfister. Causalarmor: Efficient indirect prompt injection guardrails via causal attribution. *arXiv preprint arXiv:2602.07918*, 2026.
- T. Korbak, M. Balesni, E. Barnes, Y. Bengio, J. Benton, J. Bloom, M. Chen, A. Cooney, A. Dafoe, A. Dragan, et al. Chain of thought monitorability: A new and fragile opportunity for ai safety. *arXiv preprint arXiv:2507.11473*, 2025.
- P. Kumar, D. Jain, A. Yerukola, L. Jiang, H. Beniwal, T. Hartvigsen, and M. Sap. Polyguard: A multilingual safety moderation tool for 17 languages. In *Second Conference on Language Modeling*, 2025. URL <https://openreview.net/forum?id=wbAWKXNeQ4>.
- J. Lee, F. Chen, S. Dua, D. Cer, M. Shanbhogue, I. Naim, G. H. Ábrego, Z. Li, K. Chen, H. S. Vera, et al. Gemini embedding: Generalizable embeddings from gemini. *arXiv preprint arXiv:2503.07891*, 2025a.
- K.-i. Lee, J. Koo, S. Yoon, M. Kim, H. Koh, D. Lee, and K. Jung. Program synthesis via test-time transduction. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025b. URL <https://openreview.net/forum?id=YJx8AofTF5>.
- J. Leng, Y. Liu, L. Zhang, R. Hu, Z. Fang, and X. Zhang. From static to adaptive: immune memory-based jailbreak detection for large language models. *arXiv preprint arXiv:2512.03356*, 2025.
- H. Li, X. Liu, N. Zhang, and C. Xiao. Piguard: Prompt injection guardrail via mitigating overdefense for free. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 30420–30437, 2025.
- Y. Liu, H. Gao, S. Zhai, J. Xia, T. Wu, Z. Xue, Y. Chen, K. Kawaguchi, J. Zhang, and B. Hooi. Guardreasoner: Towards reasoning-based LLM safeguards. In *ICLR 2025 Workshop on Foundation Models in the Wild*, 2025. URL <https://openreview.net/forum?id=5evTkMBwJA>.
- W. Luo, S. Dai, X. Liu, S. Banerjee, H. Sun, M. Chen, and C. Xiao. Agrail: A lifelong agent guardrail with effective and adaptive safety detection. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8104–8139, 2025.
- K. Min, M. Kim, K.-i. Lee, S. Yoon, and K. Jung. Reflectcap: Detailed image captioning with reflective memory. *arXiv preprint arXiv:2604.12357*, 2026.
- N. Miresghallah, H. Kim, X. Zhou, Y. Tsvetkov, M. Sap, R. Shokri, and Y. Choi. Can llms keep a secret? testing privacy implications of language models via contextual integrity theory. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=gmg7t8b4s0>.
- H. Nissenbaum. Privacy as contextual integrity. *Wash. L. Rev.*, 79:119, 2004.
- S. Ouyang, J. Yan, I.-H. Hsu, Y. Chen, K. Jiang, Z. Wang, R. Han, L. Le, S. Daruki, X. Tang, V. Tirumalashetty, G. Lee, M. Rofouei, H. Lin, J. Han, C.-Y. Lee, and T. Pfister. Reasoningbank: Scaling agent self-evolving with reasoning memory. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=jL7fwchScm>.

- S. Pichai, D. Hassabis, and K. Kavukcuoglu. A new era of intelligence with gemini 3. *Google*. URL: <https://blog.google/products-and-platforms/products/gemini/gemini-3>, 2025.
- Z. Ravichandran, D. Snyder, A. Robey, H. Hassani, V. Kumar, and G. J. Pappas. Contextual safety reasoning and grounding for open-world robots. *arXiv preprint arXiv:2602.19983*, 2026.
- D. Sanyal, U. Maharana, Y. Sinha, H. M. Tan, S. Karande, M. Kankanhalli, and M. Mandal. Orgaccess: A benchmark for role based access control in organization scale llms. *arXiv preprint arXiv:2505.19165*, 2025.
- Y. Shao, T. Li, W. Shi, Y. Liu, and D. Yang. Privacylens: Evaluating privacy norm awareness of language models in action. *Advances in Neural Information Processing Systems*, 37:89373–89407, 2024.
- N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36:8634–8652, 2023.
- V. Smye, V. Josewski, and E. Kendall. Cultural safety: An overview. *First Nations, Inuit and Métis Advisory Committee*, 1:28, 2010.
- M. Soltani, T. B. Moghaddam, M. R. Karim, and N. R. Sulong. The safety performance of guardrail systems: review and analysis of crash tests data. *International Journal of Crashworthiness*, 18(5): 530–543, 2013.
- L. Tamang, M. R. Bouadjenek, R. Dazeley, and S. Aryal. Handling out-of-distribution data: A survey. *IEEE Transactions on Knowledge and Data Engineering*, 2025.
- Z. Tan, J. Yan, I.-H. Hsu, R. Han, Z. Wang, L. Le, Y. Song, Y. Chen, H. Palangi, G. Lee, et al. In prospect and retrospect: Reflective memory management for long-term personalized dialogue agents. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8416–8439, 2025.
- B. Wickramasinghe, G. Saha, and K. Roy. Continual learning: A review of techniques, challenges, and future directions. *IEEE Transactions on Artificial Intelligence*, 5(6):2526–2546, 2023.
- Y. Wu, J. Guo, D. Li, H. P. Zou, W.-C. Huang, Y. Chen, Z. Wang, W. Zhang, Y. Li, M. Zhang, et al. Psg-agent: Personality-aware safety guardrail for llm-based agents. *arXiv preprint arXiv:2509.23614*, 2025a.
- Y. Wu, E. Sun, K. Zhu, J. Lian, J. Hernandez-Orallo, A. Caliskan, and J. Wang. Personalized safety in LLMs: A benchmark and a planning-based agent approach. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025b. URL <https://openreview.net/forum?id=Gsi42ohBoM>.
- Z. Xiang, L. Zheng, Y. Li, J. Hong, Q. Li, H. Xie, J. Zhang, Z. Xiong, C. Xie, C. Yang, D. Song, and B. Li. Guardagent: Safeguard LLM agents via knowledge-enabled reasoning. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=2nBcjCZrrP>.
- S. Yao, N. Shinn, P. Razavi, and K. R. Narasimhan. tau-bench: A benchmark for tool-agent-user interaction in real-world domains. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=roNSXZpUDN>.
- R. Yi, O. Suciu, A. Gascon, S. Meiklejohn, E. Bagdasarian, and M. Gruteser. Privacy reasoning in ambiguous contexts. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=0ZnXGzLcOg>.

- Y. Yuan, W. Jiao, W. Wang, J.-t. Huang, J. Xu, T. Liang, P. He, and Z. Tu. Refuse whenever you feel unsafe: Improving safety in llms via decoupled refusal training. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3149–3167, 2025.
- W. Zeng, Y. Liu, R. Mullins, L. Peran, J. Fernandez, H. Harkous, K. Narasimhan, D. Proud, P. Kumar, B. Radharapu, et al. Shieldgemma: Generative ai content moderation based on gemma. *arXiv preprint arXiv:2407.21772*, 2024.
- W. Zhang, Y. Mohamed, B. Ghanem, P. Torr, A. Bibi, and M. Elhoseiny. Continual learning on a diet: Learning from sparsely labeled streams under constrained computation. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=Xvfz8NHmCj>.
- L. Zheng, R. Wang, X. Wang, and B. An. Synapse: Trajectory-as-exemplar prompting with memory for computer control. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=Pc8AU1aF5e>.

A. Limitations and practical implications

We close by clarifying the scope of our empirical evidence and the practical implications for deploying LiSA beyond the controlled benchmark setting.

Benchmark simulation as a controlled deployment proxy. Our evaluation uses benchmark-based deployment simulations rather than logs from a live deployed guardrail. This design enables controlled, reproducible comparisons while preserving key deployment constraints: sparse feedback, corrections only for each method’s own mistakes, periodic memory refresh rather than repeated fine-tuning, and held-out evaluation never used for adaptation. Still, simulations cannot capture all properties of live deployments, where reports may be delayed, correlated, systematically noisy, or shaped by evolving organizational norms and user expectations. Our benchmarks are also English-language and focused on privacy- and safety-sensitive decisions, leaving multilingual (Kumar et al., 2025), culturally heterogeneous (Smye et al., 2010), and domain-specific settings to future work. We therefore view the experiments as controlled evidence for LiSA’s adaptation mechanism under deployment-like constraints, rather than as a claim that LiSA is the universally optimal strategy. We encourage practitioners to tailor LiSA to their own deployment settings by adapting our components to local operational constraints.

Threshold calibration as practical guidance. The default threshold $\tau_{\text{refuse}} = \tau_{\text{allow}} = 0.55$ should be understood in this spirit. Although the value is only slightly above chance, LiSA applies it to the lower 5% posterior quantile of $\text{Beta}(1 + s_m, 1 + c_m)$ for broad policy items, so it imposes a nontrivial evidence requirement. A broad policy with no contradictions is not surfaced after one, two, three, or four supports, and first passes the threshold after about five contradiction-free supports. If contradictions are observed, more evidence is required; for example, roughly $(s_m, c_m) = (7, 1), (9, 2), (11, 3)$, or $(15, 5)$ is needed to pass. Thus, the symmetric threshold $\tau_{\text{refuse}} = \tau_{\text{allow}} = 0.55$ blocks one-off broad memories and weakly supported broad policies while still allowing adaptation once modest evidence accumulates. We recommend it only as a conservative starting point, not as a deployment-independent default: the threshold should be calibrated to the target application, the expected feedback quality, and the relative cost of false accepts versus false refusals.

Asymmetric thresholds as a deployment interface. While LiSA supports asymmetric thresholds for refusal-oriented and allow-oriented broad memory, and Proposition B.3 formalizes the resulting separate posterior error budgets, we do not claim to demonstrate calibrated control over the false-accept/false-refuse trade-off in this work. In our benchmark simulations, surviving broad policy items often became high-confidence after enough evidence accumulated, making asymmetric threshold sweeps less diagnostic over the 5–10 day horizon. We therefore retain asymmetric thresholds as a practical deployment interface rather than a quantitatively validated control: real-world environments are likely to exhibit more heterogeneous confidence profiles, noisier feedback, and application-specific costs for over-acceptance and over-refusal, where allocating separate posterior error budgets to refusal-oriented and allow-oriented broad memory may be useful.

Scope of formal results. Our formal results should also be read as component-level design support rather than end-to-end guarantees. Proposition B.1 motivates allocating local refinement to mixed-label regions, while Proposition B.2 justifies confidence-gated broad reuse as an evidence-sensitive surfacing rule. Neither result models the full prompted guardrail, retrieval dynamics, or the closed online–offline feedback loop. The empirical gains in Sections 5.1–5.4 therefore reflect the joint

behavior of LiSA's memory mechanism and the underlying inference model, while the propositions clarify why the individual memory operations are reasonable.

B. Formal results and proofs

B.1. Notation

Let $\phi_B : \mathcal{X} \rightarrow \mathcal{Z}$ denote the coarse reuse state induced by broad memory and $\phi_{BL} : \mathcal{X} \rightarrow \mathcal{U}$ the refined reuse state induced after adding conflict-aware local splits, so there exists a measurable map T with $\phi_B = T \circ \phi_{BL}$. Write

$$\begin{aligned} Z &= \phi_B(X), & U &= \phi_{BL}(X), \\ \eta_B(z) &= \Pr(Y = 1 \mid Z = z), & \eta_{BL}(u) &= \Pr(Y = 1 \mid U = u). \end{aligned}$$

Let $g(p) = \min\{p, 1 - p\}$, so that

$$R_{\phi_B}^* = \mathbb{E}[g(\eta_B(Z))], \quad R_{\phi_{BL}}^* = \mathbb{E}[g(\eta_{BL}(U))].$$

For a broad state z , $\Delta(z)$ denotes the reduction in Bayes 0–1 risk attainable by any measurable refinement supported on $\{Z = z\}$.

B.2. Conflict mass bound on refinement gain

Proposition B.1 (Conflict mass bounds attainable refinement gain). For every broad state z ,

$$\Delta(z) \leq \Pr(Z = z) \cdot \min\{\eta_B(z), 1 - \eta_B(z)\},$$

with $\Delta(z) = 0$ on pure states. Under a unit-cost state-level refinement model with independent budgets, ranking broad states by the conflict mass

$$\Pr(Z = z) \min\{\eta_B(z), 1 - \eta_B(z)\}$$

maximizes the attainable upper bound on total risk reduction.

Proof. Fix a broad state z . For any refinement of $\{Z = z\}$ into sub-states,

$$\mathbb{E}[g(\eta_{BL}(U)) \mid Z = z] \geq 0,$$

so

$$\Delta(z) \leq g(\eta_B(z)) \cdot \Pr(Z = z) = \Pr(Z = z) \cdot \min\{\eta_B(z), 1 - \eta_B(z)\},$$

which proves the first claim. In particular, $\Delta(z) = 0$ when $\eta_B(z) \in \{0, 1\}$.

For the ranking statement, consider a refinement model in which (i) each broad state can be refined independently at unit cost, and (ii) refinement of one state does not affect attainable gain on any other state. Under these assumptions, the attainable total risk reduction over a budget B is bounded above by

$$\sum_{z \in S} \Pr(Z = z) \min\{\eta_B(z), 1 - \eta_B(z)\},$$

where S is the set of refined states. Selecting the top B states by conflict mass maximizes this upper bound, since the problem reduces to selecting the top B nonnegative summands. $\square$

The optimality statement is with respect to the attainable upper bound under the stated model. It should be interpreted as a ranking criterion rather than an absolute guarantee: cost structures or feasibility constraints that couple refinements across states may alter the optimal allocation.

B.3. Corollary: standard refinement inequality

Proposition B.1 recovers the classical refinement inequality as a corollary. Since $\phi_B = T \circ \phi_{BL}$, we have $\eta_B(Z) = \mathbb{E}[\eta_{BL}(U) \mid Z]$. Concavity of g and Jensen's inequality give

$$g(\eta_B(Z)) \geq \mathbb{E}[g(\eta_{BL}(U)) \mid Z],$$

and taking expectations yields

$$R_{\phi_{BL}}^* \leq R_{\phi_B}^*.$$

The inequality is strict on any positive-measure broad state that the refinement splits across the Bayes boundary $1/2$; these are precisely the states with strictly positive conflict mass. Proposition B.1 therefore localizes this classical fact by attributing attainable gain to conflict mass at the state level.

B.4. Posterior surfacing guarantee

Proposition B.2 (Posterior surfacing guarantee). Let q be a broad policy item and

$$\text{Conf}(q) = Q_\delta(\text{Beta}(1 + s_q, 1 + c_q)).$$

Under the gating rule $\text{Conf}(q) \geq \tau$, every surfaced broad item satisfies

$$\Pr(\theta_q < \tau \mid s_q, c_q) \leq \delta, \quad \mathbb{E}[1 - \theta_q \mid s_q, c_q, q \text{ surfaced}] \leq (1 - \tau) + \delta.$$

Proof. Let $L_\delta(s, c)$ denote the lower δ -quantile of $\text{Beta}(1 + s, 1 + c)$, so $\text{Conf}(q) = L_\delta(s_q, c_q)$. By the definition of the lower quantile,

$$\Pr(\theta_q < L_\delta(s_q, c_q) \mid s_q, c_q) \leq \delta.$$

Whenever q is surfaced, $L_\delta(s_q, c_q) \geq \tau$, hence

$$\Pr(\theta_q < \tau \mid s_q, c_q) \leq \Pr(\theta_q < L_\delta(s_q, c_q) \mid s_q, c_q) \leq \delta.$$

For the expected-error bound, surfacing is (s_q, c_q) -measurable, so

$$\mathbb{E}[1 - \theta_q \mid s_q, c_q, q \text{ surfaced}] = \mathbb{E}[1 - \theta_q \mid s_q, c_q].$$

Splitting on $\{\theta_q \geq \tau\}$ and $\{\theta_q < \tau\}$ and using $1 - \theta_q \leq 1 - \tau$ on the first event and $1 - \theta_q \leq 1$ on the second,

$$\mathbb{E}[1 - \theta_q \mid s_q, c_q] \leq (1 - \tau) \cdot \Pr(\theta_q \geq \tau \mid s_q, c_q) + \Pr(\theta_q < \tau \mid s_q, c_q) \leq (1 - \tau) + \delta. \quad \square$$

B.5. Corollary: label-sensitive thresholds as separate error budgets

Proposition B.3 (Error-budget interpretation). Under the gating rule of Section 3.4, every surfaced broad refusal-oriented item ($\ell_q = 1$) satisfies

$$\Pr(\theta_q < \tau_{\text{refuse}} \mid s_q, c_q) \leq \delta, \quad \mathbb{E}[1 - \theta_q \mid s_q, c_q, q \text{ surfaced}] \leq (1 - \tau_{\text{refuse}}) + \delta,$$

and every surfaced broad allow-oriented item ($\ell_q = 0$) satisfies the same bounds with τ_{refuse} replaced by τ_{allow} . Setting $\tau_{\text{refuse}} \neq \tau_{\text{allow}}$ therefore allocates independent posterior error budgets to refusal-oriented and allow-oriented broad memory.

Proof. Immediate from Proposition B.2 applied separately within each label class. $\square$

The quantity $1 - \tau_{\text{refuse}}$ upper bounds the posterior expected error of surfaced broad refusal-oriented memory, and analogously for allow-oriented broad memory. Operators prioritizing avoidance of over-refusal can raise τ_{refuse} , while those prioritizing avoidance of over-acceptance can raise τ_{allow} ; the two budgets are set independently.

B.6. Adaptive tightness: Beta vs. Hoeffding

Proposition B.2 holds for any valid lower credible bound, but the specific choice of the Beta-posterior quantile governs how quickly broad items separate from the threshold in deployment. Write $\hat{\theta}_n = s/(s + c)$ with $n = s + c$, and let

$$L_\delta^\beta(s, c) = Q_\delta(\text{Beta}(1 + s, 1 + c)), \quad L_\delta^H(s, c) = \hat{\theta}_n - \sqrt{\frac{\log(1/\delta)}{2n}}.$$

A standard quantile approximation gives, for fixed $\delta \in (0, 1/2)$ and large n ,

$$L_\delta^\beta(s, c) - \hat{\theta}_n \approx -z_{1-\delta} \sqrt{\frac{\hat{\theta}_n(1 - \hat{\theta}_n)}{n}}, \quad L_\delta^H(s, c) - \hat{\theta}_n = -\sqrt{\frac{\log(1/\delta)}{2n}},$$

where $z_{1-\delta}$ is the $(1 - \delta)$ -quantile of the standard normal distribution.

The Beta lower bound therefore scales with the sample variance $\hat{\theta}_n(1 - \hat{\theta}_n)$, which shrinks as $\hat{\theta}_n \rightarrow 0$ or $\hat{\theta}_n \rightarrow 1$, whereas the Hoeffding lower bound uses the worst-case variance $1/4$ and depends only on n . As a consequence, broad items with empirical reliability close to 0 or 1, corresponding to clearly unreliable or clearly reliable broad memory, separate from the surfacing threshold faster under the Beta score, while broad items with $\hat{\theta}_n \approx 1/2$ remain cautious under both. In this sense, the Beta choice adapts conservatism to the empirical reliability of each broad item.

B.7. Monotonicity and scope

Remark B.1 (Monotonicity). $\text{Conf}(q) = L_\delta^\beta(s_q, c_q)$ is nondecreasing in s_q and nonincreasing in c_q . Two broad items with the same empirical mean but different total evidence therefore receive different confidence scores, with more evidence producing a tighter lower bound.

Remark B.2 (Scope). Proposition B.1 concerns the representation-level Bayes risk attainable under refinement and does not model the prompt-level behavior of the deployed inference model. Proposition B.2 is a statement about the posterior reliability of a surfaced broad item conditional on its accumulated evidence; it does not capture selection dynamics across the full memory, since post-surfacing evidence depends in part on earlier broad-memory gating decisions. The periodic offline refresh described in Section 3.6 partially mitigates this by re-evaluating broad items as the report bank grows. We therefore interpret these results as supporting each component of the method in isolation, rather than as end-to-end guarantees about the full adaptive system.

C. Experiments and implementation details

C.1. Deployment simulation and splits

All experiments use the three benchmarks reported in the main text: PrivacyLens+, ConFaide+, and AgentHarm. The “+” suffix for PrivacyLens+ and ConFaide+ denotes the expanded versions used in Yi et al. (2025), which augment the original datasets with additional ambiguous-context variants. We map each dataset into the shared binary label space ALLOW/REFUSE, implemented as appropriate/inappropriate. Splits are group-preserving: when multiple rows are derived from the same base scenario, they are assigned together so that adaptation examples and held-out evaluation examples do not share the same scenario group.

For lifelong adaptation, each method receives a stream of deployment queries and is evaluated repeatedly on a fixed held-out set. On each day, a method first predicts labels for that day’s streamed

cases. At day end, it receives only the cases it misclassified, paired with the reported label, and updates its memory before the next day. The fixed held-out set is never used for adaptation and is evaluated after each daily update. Unless otherwise stated, results are averaged over five seeds.

Table 4 | **Deployment simulation sizes.** Only misclassified streamed cases are reported to the adaptive method at day end. Held-out examples are fixed across days and are never used for adaptation.

Benchmark	Daily stream	Days	Held-out evaluation
PrivacyLens+	50 queries/day	10	500 examples
ConFaide+	50 queries/day	10	500 examples
AgentHarm	60 queries/day	5	116 examples

AgentHarm uses a shorter deployment horizon because fewer examples are available after constructing a group-preserving held-out split. For noisy-feedback experiments on AgentHarm, we use a fixed 200-example stream and apply label flips only to reported failures, following the same corruption protocol described below.

C.2. Model and retrieval configuration

The online guardrail is either Gemini-3.1-flash-lite or Claude-Haiku-4.5. The offline manager for reflection and policy induction is Gemini-3.1-pro, and retrieval uses Gemini-embedding-001. All generation calls use temperature 0 and deployed with Google Cloud Vertex AI ¹. At inference time, retrieved context is bounded: methods retrieve at most five similar cases and two policy-like memory items per memory type. For LiSA, the final prompt contains at most two retrieved local rules and two confidence-filtered broad policy items. All model inference, offline policy-induction, and embedding calls were served through managed Google Cloud Vertex AI APIs; we did not run local model inference or allocate local GPU workers, so hardware details such as worker type and memory were abstracted by the managed API service.

C.3. Policy gating and noise construction

LiSA applies the Beta lower-credible-bound confidence score described in Section 3.4 to broad policy items only, with prior Beta(1, 1) and $\delta = 0.05$. Unless otherwise stated, both label-specific thresholds are set to $\tau_{\text{refuse}} = \tau_{\text{allow}} = 0.55$. If no local rule is retrieved and no retrieved broad policy item passes the threshold, LiSA does not prompt with empty memory; it falls back to the base guardrail.

For noisy-feedback experiments, only reported failure labels are corrupted. Each reported label is independently flipped with probability ρ , and the corrupted label is then used by the adaptation method. The held-out labels used for evaluation are never corrupted.

C.4. LiSA offline refresh

At the end of each deployment day, LiSA refreshes memory from the newly reported failures and the accumulated case history. The refresh has two memory channels. First, newly reported failures are passed to Template 2 to induce broad structured preventive items. These items are stored as general policy candidates with a recommended label and initial support/contradiction counts from the reported batch. Second, LiSA rebuilds conflict-aware local rules from the accumulated case

¹<https://cloud.google.com/vertex-ai>

memory. This step clusters semantically similar cases, keeps mixed-label neighborhoods, and creates narrow label-specific rules for the local boundary.

The broad-policy induction step uses the following deterministic grouping and merging procedure around the LLM synthesis call. All failures newly reported at the end of the current deployment day form one induction group; in our experiments this group contains only the model’s misclassified streamed cases for that day. Template 2 is then called once per non-empty group with temperature 0 and is allowed to return at most three structured preventive items. Each item is parsed into Title, Description, Content, Recommended label, and Rule type; invalid or missing labels are resolved to the majority corrected label in the inducing group. The item’s provenance is the full set of report ids in the group, its label-skew metadata is the corrected-label histogram of the group, its initial support count is the number of inducing reports whose corrected label matches the item’s recommended label, and its initial contradiction count is the remaining number of reports in that group.

Across refreshes, broad items are not appended as independent rules forever. We keep the raw induced candidates and rebuild the broad memory by embedding each canonical structured statement with Gemini-embedding-001, then clustering statements with agglomerative clustering using cosine distance, average linkage, no fixed number of clusters, and distance threshold 0.20. For each cluster, the representative statement is the member whose embedding has maximum cosine similarity to the cluster centroid. Cluster metadata is obtained by summing support counts, contradiction counts, label-skew histograms, and representative report ids over members. LiSA does not use a second LLM call to rewrite merged clusters; the representative statement is reused verbatim. This makes the cross-day grouping criterion exactly the embedding-cluster membership of induced statements, rather than an unreported manual taxonomy.

Mixed-label regions are detected separately from the broad-policy clusters, using the accumulated case memory rather than LLM-generated policy text. Cases are embedded from their canonical scenario summaries and clustered within each domain namespace with agglomerative clustering using cosine distance, average linkage, no fixed number of clusters, distance threshold 0.20, and minimum cluster size 2. A cluster is marked as mixed-label if its corrected labels contain both ALLOW/appropriate and REFUSE/inappropriate; we do not impose an additional balance threshold beyond at least one case of each label, and record the conflict score as $1 - \max_y n_y / \sum_y n_y$. Pure clusters are discarded.

The local-rule text is rendered deterministically from the mixed cluster. We form a region summary by linearizing the case metadata already present in the input records, and identify decisive pivots as the attributes or textual facets whose common values differ across the ALLOW/appropriate and REFUSE/inappropriate members of the same cluster. Thus, local memory is generated only from semantic neighborhoods that contain both labels; the input fields only determine how the already-detected boundary is verbalized, rather than serving as hand-written benchmark-specific decision rules.

Existing broad policies are not assigned a separate LLM update prompt. If a broad policy was surfaced during inference, its evidence is updated directly: a label match increments support and a mismatch increments contradiction. During refresh, all raw broad policy candidates are re-clustered by embedding similarity, similar policies are merged by aggregating support and label skew, and runtime statistics are carried over only for surviving broad policy text. A broad policy is treated as near a conflict-heavy region when its embedding has cosine similarity at least 0.85 to a mixed-label conflict entry. Low-confidence broad items remain stored but are not surfaced unless their confidence later exceeds the label-specific threshold.

The two channels play different roles at runtime. Broad policies provide reusable default guidance

induced from sparse failures. Local rules act as narrow warning or exception cues in regions where nearby cases split labels. Both are retrieved by semantic similarity. Broad policies are confidence-filtered before serialization, while retrieved local rules are serialized directly as narrow cues.

C.5. Baselines implementation

All baselines use the same binary decision task and JSON label interface as the base guardrail. We do not reproduce every baseline prompt verbatim, since the primary experimental distinction across methods is the memory object each baseline maintains and retrieves. Where a baseline was originally designed for a different supervision regime, we describe the adaptation explicitly below. Pure Prediction uses the base guardrail prompt alone. AGrail retrieves adaptive checklist-style notes and generates a short runtime checklist before classification. Synapse retrieves similar past decision records as case-level exemplars. ReasoningBank retrieves reusable natural-language memories induced from prior decision outcomes.

Synapse and ReasoningBank were originally designed for long-horizon agent tasks rather than single-step binary guardrail decisions, so we adapt them to the feedback interface studied in this work. For Synapse, trajectory exemplars are replaced by guardrail decision records consisting of the scenario, the model prediction, and the corrected label. This gives a case-memory baseline that tests whether adaptation can be achieved by retrieving similar reported failures directly, without inducing policy-level abstractions.

For ReasoningBank, the original trajectory-level memory construction is not directly applicable in our setting. ReasoningBank induces reasoning memories from success and failure trajectories, and in long-horizon tool-use tasks can extract multiple reflections from different parts of a trajectory, including individual tool-call decisions. By contrast, our deployment-time guardrail interface provides each adaptive method only with sparse misclassified cases and their corrected ALLOW/REFUSE labels; it does not expose successful trajectories, dense labels for the full stream, or tool-call-level supervision. Instantiating the full ReasoningBank pipeline would therefore require additional information unavailable to the other baselines. We consequently implement ReasoningBank as a ReasoningBank-style broad-policy memory baseline: reported failures are converted into reusable natural-language policy memories and retrieved as decision-boundary guidance inside the same binary classification prompt. We do not add LiSA’s conflict-aware local memory or confidence-gated broad-policy surfacing to this baseline. This implementation matches the broad-policy-only role of LiSA without local rules or confidence-gated surfacing, making it both an external memory baseline adapted from prior work and a controlled reference point for isolating the contribution of LiSA’s two additional mechanisms.

Under this adaptation, the baselines form a natural memory ablation hierarchy: Pure Prediction removes memory entirely, Synapse tests raw case reuse, ReasoningBank tests broad policy abstraction under the same sparse failure-report interface, and LiSA tests whether adding conflict-aware local memory and conservative confidence-gated broad-policy reuse improves over broad abstraction alone.

C.6. LiSA prompt templates

LiSA uses fixed templates. The two templates below are the only LLM prompts introduced by LiSA beyond the shared base guardrail decision prompt. Indexed fields such as Case {i}, Local Rule {i}, Memory {i}, and Failure {i} denote repeated blocks; each prompt contains all retrieved or reported items up to the limits described above. Confidence updates are not prompted: after a surfaced broad item receives feedback, its support or contradiction count is updated by exact label match, and the Beta lower credible bound is recomputed. Existing policies are not rewritten by a separate LLM update prompt during refresh; newly reported failures are converted into additional

structured items using Template 2, while policy merging and local-rule construction are deterministic procedures over embeddings, label counts, and stored policy metadata.

The inference template is instantiated with the current scenario, retrieved similar cases, retrieved local rules, and confidence-filtered broad policy items. If no local rule is retrieved and no broad item survives filtering, this template is not used and the system falls back to the base guardrail.

Prompt Template 1: LiSA Inference with Applicability Checks

```

Relevant similar past cases:
  Case {i}: {case_summary}
  Outcome: {case_label}

Cases with semantic similarity despite conflicting labels:
  Content {i}: {canonical_policy}

Structured preventive memory:
  Memory {i}:
    Title: {title}
    Description: {description}
    Content: {content}
    Type: {rule_type}, label={recommended_label}, confidence={confidence}

Given the following new scenario, determine whether the action is
appropriate or inappropriate from a safety or privacy perspective.
When applicable, prioritize high-confidence structured preventive
memories over lower-confidence or less specific retrieved context.

Before the final answer, explicitly check each retrieved memory item for
applicability. For each item, decide one of: apply, skip, override, or
uncertain. Use apply when the item clearly matches the case. Use skip
when it is not a real fit. Use override only when a narrow exception-level
cue should dominate a broader rule. Use uncertain when the fit is
ambiguous. Then make the final decision using only the items that
survived this applicability check.

Scenario:
{scenario_text}

Respond with JSON only in this order:
{"reasoning": "briefly list memory applicability decisions, then final
  rationale",
  "label": "appropriate" or "inappropriate"}
```

The offline policy-induction template converts day-end failures into compact structured memory. The same template is used for broad preventive items; local rules are then regenerated from mixed-label neighborhoods during the memory refresh.

Prompt Template 2: Offline Preventive Memory Induction

Reported failures to convert into preventive memory:

```
Failure {i}:
  Scenario: {scenario_text}
  Model prediction: {predicted_label}
  Correct label: {true_label}
```

Convert these failures into concise, generalizable preventive memory items. Avoid quoting case-specific names or wording. Keep each item compact and reusable. Produce at most 3 items.

Each item must follow exactly this multiline format:

```
Title: short risk pattern title
Description: one-line summary
Content: preventive rule, decisive boundary, and when to defer if needed
Recommended label: appropriate or inappropriate
Rule type: general_policy or local_exception
```

```
Respond with JSON: {"insights": ["item1", "item2", ...],
  "policies": ["item3", ...]}
```

The Rule type field in Template 2 is metadata attached to LLM-induced preventive items. LiSA’s conflict-aware local rules are constructed separately from mixed-label neighborhoods rather than generated by this prompt. For clarity, the deterministic local-rule rendering format is shown below.

Deterministic Template for Conflict-Aware Local Rule

```
Local rule: In the boundary-heavy region '{region}', {recommended_outcome}.
Evidence: support={support_count}, nearby contradictions={contradict_count}.
Decisive pivots: {pivot_1}; {pivot_2}.
Use this as a narrow exception-level cue when the current case matches the
same local pattern.
```

C.7. Existing assets and licenses

We use publicly available datasets and commercial API models. Table 5 summarizes access paths and licensing terms. We cite the original papers in the main text and use each asset in accordance with its released license or API terms of service. For Claude-Haiku-4.5, we use this model through Google Vertex AI. AgentHarm restricts use to research that improves the safety and security of AI systems; our use of the benchmark for evaluating safety guardrails is consistent with this restriction.

D. Additional discussion and case study**D.1. Offline adaptation cost**

The cost-performance analysis in Section 5.4 reports runtime latency per guarded decision because this cost is paid on every deployment input. LiSA additionally performs offline memory refresh when user-reported failures are accumulated. This cost is not on the critical inference path and can be

Table 5 | **Assets used in this work.** Datasets and models with their access paths and license/terms.

Asset	Type	Access	License / Terms
PrivacyLens (Shao et al., 2024)	Dataset	GitHub	CC BY 4.0
ConFaide (Mireshghallah et al., 2024)	Dataset	GitHub	CC BY 4.0
AgentHarm (Andriushchenko et al., 2025)	Dataset	HuggingFace	MIT (with safety-use restriction)
Gemini-3.1-pro (Pichai et al., 2025)	Model API	Google Vertex AI	Vertex AI ToS
Gemini-3-flash (Pichai et al., 2025)	Model API	Google Vertex AI	Vertex AI ToS
Gemini-3.1-flash-lite (Pichai et al., 2025)	Model API	Google Vertex AI	Vertex AI ToS
Gemini-embedding-001 (Lee et al., 2025a)	Model API	Google Vertex AI	Vertex AI ToS
Claude-Haiku-4.5 (Anthropic, 2025)	Model API	Google Vertex AI	Anthropic Usage Policy

batched across reports.

In our implementation, the offline policy-induction step consumes on average 427 input tokens and 2300 output tokens per reported failure. If a report-derived policy is reused over K subsequent guarded decisions, its amortized offline cost is

$$427/K \text{ input tokens} \quad \text{and} \quad 2300/K \text{ output tokens}$$

per decision. For example, even at $K = 100$, this corresponds to only 4.27 input tokens and 23 output tokens per decision.

Thus, offline adaptation has a different cost structure from model scaling. Scaling the base guardrail increases cost on every inference call, whereas LiSA pays a small offline cost only when sparse failures are reported and then reuses the induced policies across many later inputs. This is why the main cost-F1 comparison focuses on runtime serving cost.

D.2. Impact of offline manager quality

A natural question is whether the offline adaptation cost can be reduced by using a smaller model as the offline policy manager. This changes a different cost axis from the one measured in Section 5.4. Runtime latency is incurred on every guarded input and is determined by the online guardrail call, retrieval, and the serialized memory prompt. By contrast, the offline manager is invoked only when sparse user-reported failures are accumulated and memory is refreshed. Thus, using a weaker offline manager can reduce intermittent refresh cost, but it does not directly reduce the per-query critical inference path once the memory has been deployed.

Table 6 evaluates this trade-off by replacing the default offline manager with a weaker model while keeping the online guardrail fixed. Performance drops substantially, from 0.962 to 0.915 final-day macro-F1. This indicates that the offline manager performs substantive abstraction work: under sparse feedback, it must induce broad policies from few reports and identify boundary-sensitive local refinements in mixed-label regions, rather than merely formatting reported failures into memory entries. A weaker manager can produce brittle, underspecified, or overly broad policies, which are then reused across many subsequent online decisions.

Table 6 | Impact of offline manager quality on final-day macro-F1. Same setting as Table 1: $\rho = 0\%$, averaged across benchmarks, with Gemini-3.1-flash-lite as the online guardrail.

Offline Manager	Macro-F1
Gemini-3.1-pro (default)	0.962
Gemini-3-flash	0.941
Gemini-3.1-flash-lite	0.915

This makes offline manager quality especially important in the deployment regime studied here. If dense feedback were available after every trial, one could update memory online and correct low-quality policies quickly. In the guardrail setting, however, feedback arrives sparsely and memory is refreshed only periodically. A single low-quality offline abstraction can therefore persist and affect many future inferences. Since the offline refresh cost is outside the critical serving path and is amortized over later guarded decisions (Appendix D.1), we recommend using a strong offline manager by default while keeping the online guardrail lightweight. Weaker offline managers remain a viable option when refresh-time compute is constrained, but Table 6 should be read as the expected quality cost of that choice.

D.3. Examples of generated policies

In this section, we provide concrete examples of policy items generated by LiSA during the deployment simulation. These examples illustrate how the system induces broad general policies from sparse reports, as well as how it constructs conflict-aware local rules to handle mixed-label boundaries. Unnecessary metadata (such as internal tracking counts and IDs) have been omitted for clarity.

Example 1: Broad General Policy

Title: Sharing General or Public Information

Description: Professionals can share non-sensitive, general information regardless of their confidential roles.

Type: `general_policy`

Recommended Label: appropriate

Evidence: Support = 170, Contradictions = 2

Confidence: 0.964

Content: It is appropriate for individuals in confidential roles to share general, publicly available, or non-sensitive information (e.g., event schedules, industry conferences, general techniques) with external parties, provided no protected personal or proprietary data is disclosed.

Example 2: Broad General Policy

Title: Conflation of Secrets with Public/General Info

Description: Do not confuse a subject’s confidential secret with their public activities or general opinions.

Type: `general_policy`

Recommended Label: appropriate

Evidence: Support = 47, Contradictions = 2

Confidence: 0.879

Content: When evaluating a scenario, isolate the exact piece of information being shared. If the information is public advocacy, volunteer work, event attendance, or a general opinion, it is appropriate to share. Do not label it inappropriate just because the subject also has a related, highly sensitive secret in the background context. The boundary is whether the secret itself is being disclosed.

Example 3: Conflict-Aware Local Rules

These two complementary rules demonstrate how LiSA resolves a boundary-heavy region where cases split between appropriate and inappropriate contexts.

Local Rule 3a: Appropriate Cue

Content: In the boundary-heavy region ‘*sharing Ben attended a public lecture...*’, treat the case as appropriate.

Decisive Pivots:

- *Information type differs:* appropriate cases leaned toward public lectures and interfaith dialogue, while inappropriate cases leaned toward fringe groups with radical beliefs.
- *Sharing reason differs:* Emily expressed concern vs. Sarah shared her growing fears.

Local Rule 3b: Inappropriate Cue

Content: In the boundary-heavy region ‘*sharing Ben attended a public lecture...*’, treat the case as inappropriate.

Decisive Pivots: Inverse of the pivots above, capturing the opposing boundary conditions.